



ENSAI CINEMA CLUB

Analysis File

Étudiants :

N. Bougis,
S. Mironescu
Z. Goulamaly
S. Harris

Tuteur :
C. Valot

Summary

Introduction	2
Part 1 – Functional Analysis	2
Rephrasing the requirements	2
Analysis of the features	2
Book a screening for an upcoming movie :	2
Prepare the schedule of movie screenings :	2
Accounting details :	2
Implementation of the features	3
Priority features	3
API uses	3
Features included if time allows :	4
Features excluded :	4
Summary : Use case diagram	4
Part 2 – Technical-Functional Layout	5
Benchmark of the Movie Information API	5
Accounting Module	6
Overall structure and integration with third-party services	6
Part 3 – Technical Design	6
Data model	6
Class diagram	7
Activity diagrams	8
Part 4 – Project management description	9
Team organization	9
Task assignment	9
Methodology	10
Gantt diagrams	10
Phase 1 : Analysis (28/08 - 17/09)	10
Phase 2 : Development (17/09 - 21/11)	11
Conclusion	11

Introduction

Part 1 – Functional Analysis

Rephrasing the requirements

Three features are a must for our application, they relate to the user experience and the different functionalities needed for an administrator to operate and manage cinema reservations. Those three features are as followed :

- As a customer, I can book a screening for an upcoming movie
- As an administrator, I can prepare the schedule of movie screenings
- As an administrator, I can see accounting details for a given period

First of all, this project must have two different sides : a customer side that will be the public and an administrator side. Accounts with the correct rights for the administrators are therefore required.

Analysis of the features

For each features, here is the analysis of the requirements that need to be met in order to succesfully implement the feature.

Book a screening for an upcoming movie :

We will need to implement a time management tool in order to classify movies as upcoming or ended. Many clients may want to book a ticket to the same screening at the same time, it is then essential for us to create a special status and management for unconfirmed bookings in order to prevent the oversale of tickets. Booking implies the implementation of a payment and confirmation system.

Prepare the schedule of movie screenings :

Given information about the movies that wished to be scheduled we will implement, without a front-end, a way to add movies to each room without there being overlaps between sessions. Information about movies will be taken via an API that will be from TMDb. With this information we will be able to recover the duration of the film and establish the time at which the screening will end.

Accounting details :

Accounting details will be provided to administrators using the screening history and supplementary revenues from the other activities of the cinema. Graphs will be provided in order to have them be implemented later in the front-end by the other team.

Those three features mean, in technical terms, creating a backend that will be able to support a comprehensive frontend for customers to reach our cinema (that will be developed as a separate project). This can be translated by the existence of our personal API that will communicate information to the frontend of the application. It also means having a separate system dedicated for the administrators where the schedule can be made and information stored.

Lastly, those features imply access to an external API in order to properly set schedules and have automatic and unique information about movies to present to the customers. In order to use the limited requests of the TMDB API efficiently, we will establish a private database via POSTGRE SQL to store data about movies that will be screened. It will also be useful to track user actions (seen movies, posted comments...) in order to provide a more personal experience.

Implementation of the features

Priority features

Here are the features that are at the core of the application's design :

User features :

For the customer, we will implement different features which will mainly be having access to the scheduled movies, seeing information about said movies, and being able to book a screening. An important part of this will be the query system that will allow the customer to search screenings by name of movie, genre, length, date and time of screening (maybe more).

The user will also be able to create an account to enjoy rating and writing comments about seen movies, have a friend list and be able to interact with them. It will be possible to establish one with an email address, pseudonym and password. Implementing this will mean having to implement a database corresponding to the movies seen by the user and a secure way to add them to the seen list to avoid review bombing on the future website. Finally, a payment system will be implemented in order to have a fully functional back-end that will fit all the needs of our customer.

Admin specific features :

Administrator accounts will be the same as user accounts, allowing them to also interact socially, but will have access to more features. Those features are mainly : establishing a schedule for the different rooms, being able to modify the pricing and also have the ability to check accounting details in order to run the cinema efficiently. Accounting details will not provide official documents but statistics about what the administrator would want to know. A search per time interval, movie and genre will be implemented. A summary of the statistics we wish to implement :

- number of tickets sold
- turnover
- theatre occupancy rate
- sales per film
- sales per screening
- sales per day/week/month
- average ticket price

API uses

Movie informations :

The list of scheduled movies being quite small and the potential userflow of the website quite

high, we will import the movie data once on a personal database that will then be accessible without external constraints. TMDB's API provides the summary in french and the french name of the movie. Using an external API will allow us to provide the customer with all the information about any movie that the cinema will provide, for example its duration, actors, rating, trailer, box office, ratings and more.

Payment system :

We will implement a working payment system using Stripe. This method will allow us to implement a way to confirm bookings without having to worry about the privacy of the credit cards.

Features included if time allows :

For the moment we do not plan to implement a system to let the customer choose their seats. This method is quite common and already in place at Bruz's cinema. However, if time allows, it would be a plus that would help us provide a better experience for motivated customers.

Features excluded :

In this project, we will not allow the customers to cancel their bookings nor get a refund. We will also not send confirmation emails but rather provide the customer with a confirmation screen and access to its bookings.

Summary : Use case diagram

To summarize, here is the use case diagram :

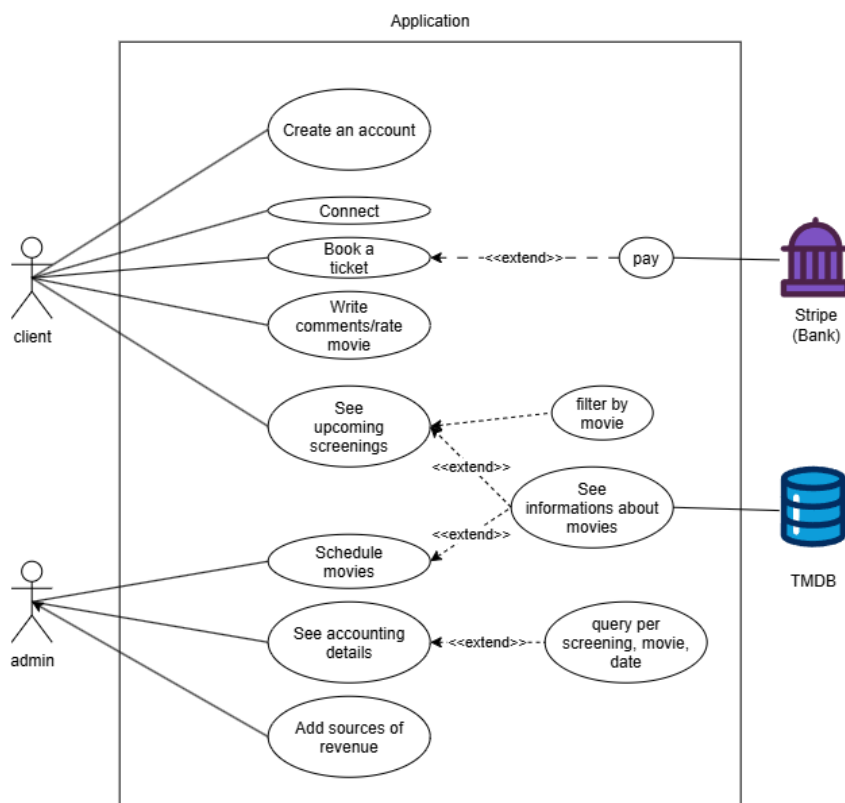


FIGURE 1 : Use case diagram : features available for users

Part 2 – Technical-Functional Layout

Benchmark of the Movie Information API

Three movie data APIs are under consideration for the project : IMDb, OMDb, and TMDB. The goal is to compare them, rule out any unsuitable options, and select the one that provides the most relevant information based on the transmitted JSON (e.g., searching for information on the movie “The Odyssey”).

Comparison of the Three APIs

Criteria	OMDb	TMDB	IMDb
Price	Free (1000 req./day)	Free (40 req./10s) or paid upon negotiation	Paid (AWS, min. \$150,000)
Official Public API	Yes	Yes	No
Ease of Use	Easy	Easy	Difficult (AWS)
Data Completeness	Complete	Very complete	Very complete
License	Commercial use allowed	Non-commercial use by default	Mandatory commercial license
Language	English only	French supported (fr-FR)	French supported

Exclusion of IMDb

IMDb ruled out : No official public API available : access is provided via AWS with a minimum entry cost of \$150,000. Incompatible with the our plan to build an app. The following evaluation therefore focuses solely on OMDb and TMDB.

OMDb vs TMDB — JSON Comparison

Based on the same search query (“The Odyssey”), here is the relevant data returned by each API :

Data Field	OMDb (JSON)	TMDB (JSON, language=fr-FR)
Title	"Title": "The Odyssey" (<i>English only</i>)	"title": "L'Odyssée" (<i>Translated into French</i>)
Synopsis	"Plot" in English only “After the Trojan War...”	"overview" translated into French “Vingt ans après son départ...”
Genres	"Genre": "Action, Adventure, Fantasy" (<i>English text</i>)	"genres": [{"id_genre": "Aventure"}, ...] (<i>Translated</i>)
Runtime	"Runtime": "172 min"	"runtime": 173 (<i>minutes</i>)

Data Field	OMDb (JSON)	TMDB (JSON, language=fr-FR)
Release Date	"Released": "17 Jul 2026"	"release_date": "2026-07-15"
Content Rating	'Rated': 'R' (<i>US age rating classification</i>)	"adult": False (<i>Indicates if the movie is rated adult/porno- graphic</i>) "softcore": False (<i>Specific flag designating non-explicit erotic content</i>)

Final Decision

TMDB is selected : It is the only candidate among the two to return key fields such as title, synopsis, and genres translated into French (via the language=fr-FR parameter), whereas OMDb strictly provides these fields in English.

Accounting Module

The Accounting Module collects sales data and tracks financial performance. This report presents the main accounting and analytics features chosen for the system.

Key Indicators

- **Number of Tickets Sold** : Shows the total volume of sales. This number measures attendance and is used for official ticketing reports.
- **Total Revenue** : The total money collected over a chosen period. It helps monitor the general financial health of the cinema.
- **Hall Occupancy Rate** : The percentage of filled seats compared to the total capacity of the room. It shows how well each room and movie schedule performs.
- **Revenue by Movie** : Shows how much money each movie makes. This is useful to check movie profitability and calculate payments for distributors.
- **Revenue by Show** : Shows the earnings for each specific showtime (afternoon, evening, early screenings) to help improve the schedule.

Time Tracking

For both Total Revenue and Number of Tickets Sold, you can filter data by selecting a start date and an end date. You can also view results by day, week, or month with a graph to identify seasonal trends and follow performance over time.

Overall structure and integration with third-party services

Part 3 – Technical Design

Data model

Here is the data model of our databases :

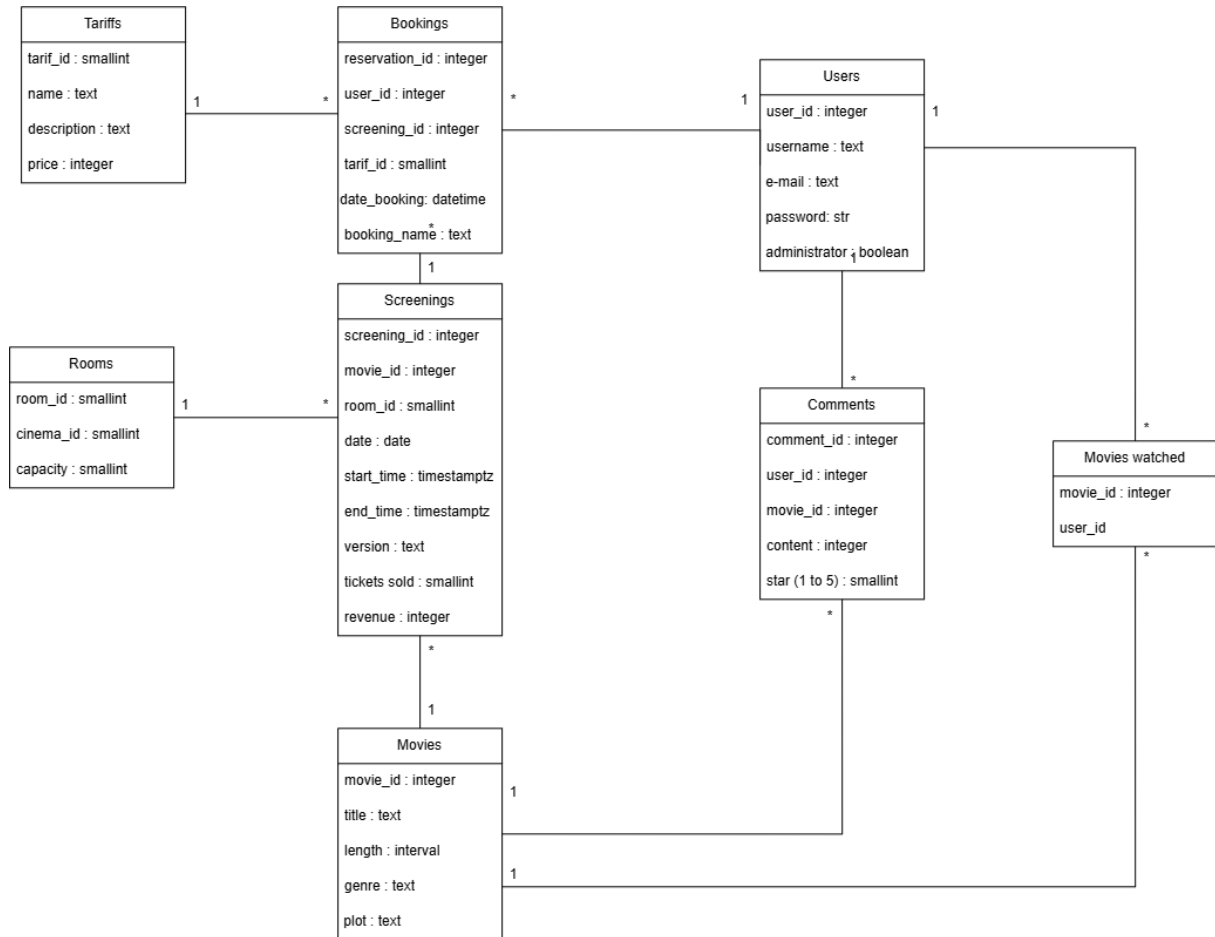


FIGURE 2 : Data model : structure of the application's database

Each database contains an exclusive id, external ids and information that will need to be provided to the different users. “Movies watched” is only to keep track of the movies each user has watched and therefore will not be turned into a python object but rather a list of objects extracted from the database. “Tariffs” and “Rooms” will allow administrators to change the tariffs provided and information about new and existing rooms.

Class diagram

Here is the class diagram of our application :

The core idea is to separate the layers of activity to improve testability and separation of concerns. We thus have business objects which are modified and created via a dedicated service class that will communicate to a dedicated DAO class in order to access the database.

Activity diagrams

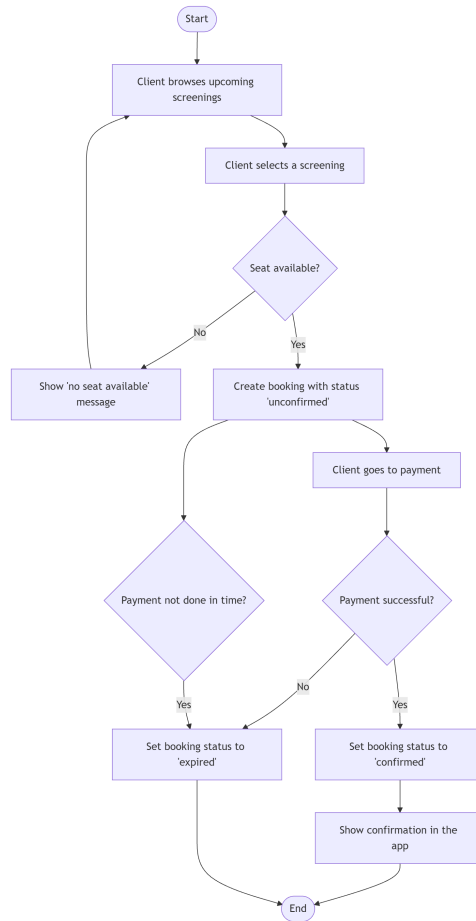


FIGURE 3 : Activity diagram : a client booking a screening

This diagram shows the booking process from the client's side : picking a screening, checking seat availability, paying, and getting a confirmation or an expired booking if the payment fails or takes too long.

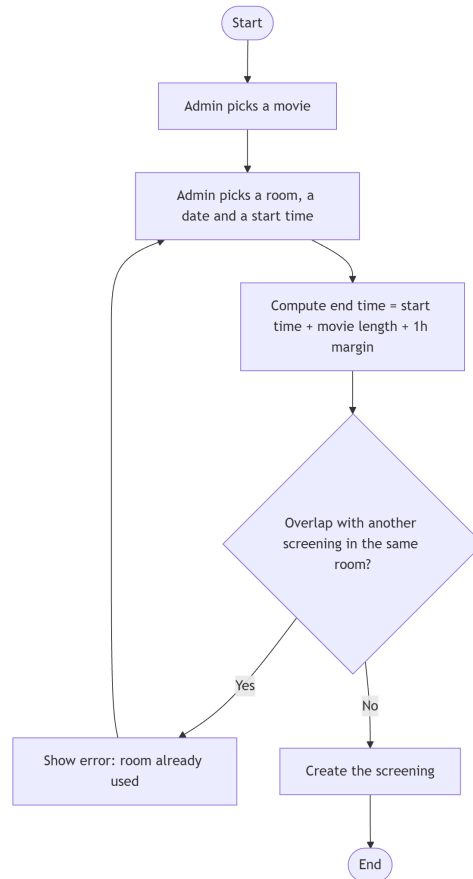


FIGURE 4 : Activity diagram : an admin scheduling a screening

This diagram shows how an admin adds a new screening to the planning. The main step is checking that the room is free, with at least 1 hour of margin before and after other screenings.

Part 4 – Project management description

Team organization

We are 4 people in the team and each one has a role for the whole project :

Role	Person
Project Manager	Nolann
Technical Director	Simon
DAO Supervisor	Samuel
Testing Supervisor	Zoeb

Task assignment

We split the work mostly along these roles. Nolann handles coordination, planning and keeping the Gantt diagram up to date. Simon takes care of the technical side, meaning the architecture, the class diagram and the general design choices. Samuel is more on the data side, so the database and the data model, and how we access the data. And Zoeb is in charge of testing, basically making sure stuff actually works before we call it done.

Of course this is not a strict separation : everyone can work on any part if needed, especially since we're only 4 people. But these roles give each of us a main area to focus on and to be responsible for.

Methodology

We have a team meeting every week to check where everyone is, discuss blocking points and decide what to do next. For now we mostly talk on WhatsApp when we need to decide something quickly, and we put our documents on Google Drive so everyone can access them. We know that WhatsApp is not the best communication tool for this sort of project work , so we're currently looking for a possibly more efficient tool, but we haven't switched yet.

The code is on GitHub (github.com/nolannb35/ENSAI-2A-projet-info-team6), we use it to keep track of the different parts of the project and work at the same time without breaking each other's code.

We also want to do some kind of review before the big deadlines (Analysis File, immersion, final delivery), just to check everyone's part actually fits together before it's too late to fix.

Gantt diagrams

Phase 1 : Analysis (28/08 - 17/09)

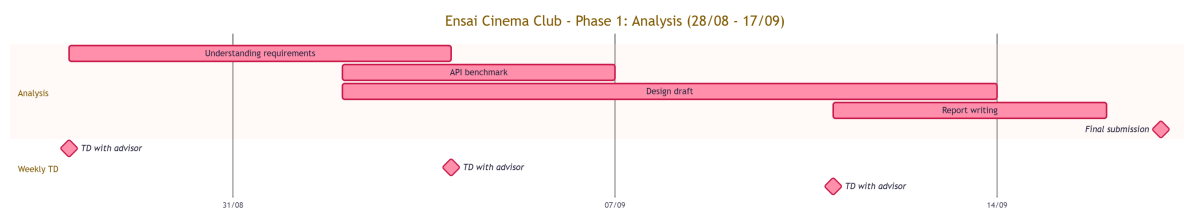


FIGURE 5 : Gantt chart for the analysis phase (28/08 - 17/09)

This phase goes from the start of the project to the Analysis File deadline. We start by understanding the subject and turning it into real features, then we do the movie API benchmark at the same time as a first draft of the technical design, and we finish by writing and reviewing the report. The three TD meetings are basically checkpoints with our advisor, where we can still adjust the plan if something's off.

Phase 2 : Development (17/09 - 21/11)

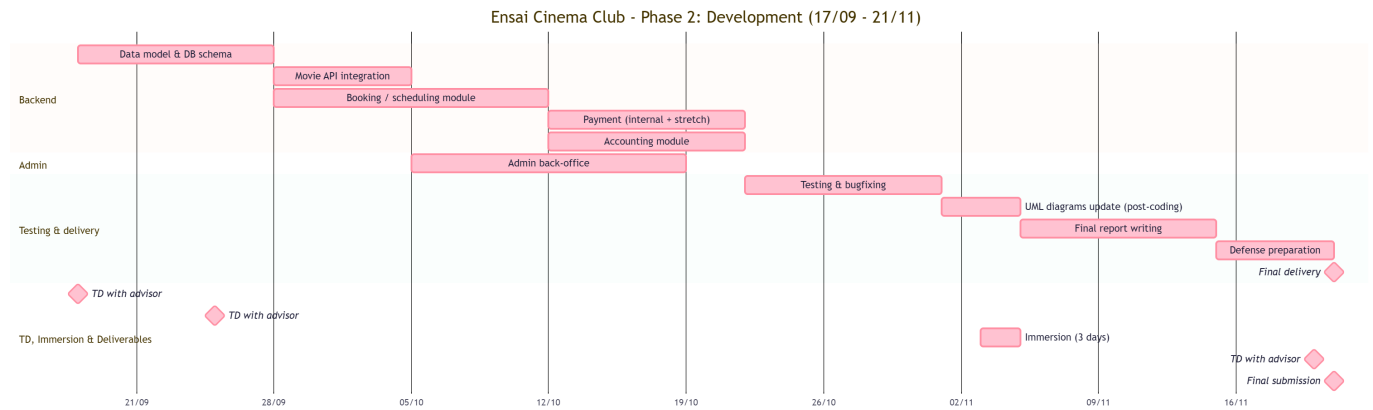


FIGURE 6 : Gantt chart for the development phase (17/09 - 21/11)

This phase is where we actually code stuff. We start with the backend (database, movie API, booking, payment, accounting), then the admin back-office, and we finish with testing, updating the UML diagrams to match what we actually coded, writing the final report and prepping the defense. The 3-day immersion in early November is when we're expecting to move a lot faster than usual since we'll have way more time on it. That's basically it, everything ends up pointing to the final submission on 21/11.

Conclusion

Annexes

Annex 1 : OMDb API Query

This annex shows a Python script using the OMDb API to search for “The Odyssey” and retrieve its details in JSON.

```

onxyia@vscode-python-570725-0:~/work$ /opt/python/bin/python "/home/onxyia/work/import requests.py"
{'Title': 'The Odyssey', 'Year': '2006', 'Rated': 'R', 'Released': '17 Jul 2006', 'Runtime': '172 min', 'Genre': 'Action, Adventure, Fantasy', 'Director': 'Christopher Nolan', 'Writer': 'Christopher Nolan, Homer', 'Actors': 'Matt Damon, Tom Holland, Anne Hathaway', 'Plot': 'After the Trojan War, Odysseus faces a dangerous voyage back to Ithaca, meeting creatures like the Cyclops Polyphemus, Sirens, and Calypso along the way.', 'Language': 'English, Greek', 'Country': 'United Kingdom, United States, Canada, New Zealand, Iceland', 'Awards': '3 wins total', 'Poster': 'https://m.media-amazon.com/images/S/MV5BNjZlcmNmZjZmOTUwYjY0NDY1TGJlNGQyYTZlY2ZuMmM3NmI1XkEYXkF6CG_v1_QL75_UX380_CR0_0,380,562_jpg', 'Ratings': [{'Source': 'Internet Movie Database', 'Value': '8.4/10'}, {'Source': 'Rotten Tomatoes', 'Value': '93%'}, {'Source': 'Metacritic', 'Value': '88/100'}, {'Source': 'Metascore': '88', 'imdbRating': '8.4', 'imdbVotes': '477,953', 'imdbID': 'tt33764258', 'Type': 'movie', 'DVD': 'N/A', 'BoxOffice': '$588,976,965', 'Production': 'N/A', 'Website': 'N/A', 'Response': 'True'}]
onxyia@vscode-python-570725-0:~/work$

```

Annex 2 : TMDb API Query

This annex shows a Python script using the TMDb API to search for “The Odyssey” and retrieve its details in JSON.

```
onyxia@vscode-python-570725-0~/.works /opt/python/bin/python "/home/onyxia/work/import requests.py"
{"adult": false, "backdrop_path": "/7uiyLzRSllGnT7B3Z2idKw0Mcq.jpg", "belongs_to_collection": null, "budget": 250000000, "genres": [{"id": 12, "name": "Adventure"}, {"id": 28, "name": "Action"}, {"id": 14, "name": "Fantastique"}], "homepage": "", "id": 1368337, "imdb_id": "tt33764258", "origin_country": ["GB", "US"], "original_language": "en", "original_title": "The Odyssey", "overview": "Vingt ans apr\u00e8s son d\u00e9part pour la guerre de Troie, le roi Ulysse rentre enfin \u00e0 Ithaque, mais son voyage est parsem\u00e9 d'aventures et d'\u00e9preuves.", "popularity": 404.6591, "poster_path": "/kP8jIkmXoWEYiq9cS2U0dvcn.jpg", "production_companies": [{"id": 33, "logo_path": "/8lvhyhjR8OUkO0y2dKxXoALWkdP.png", "name": "Universal Pictures"}, {"id": 9996, "logo_path": "/3tvBqysBhxwEHLu62SIleU9307.png", "name": "Syncope", "origin_country": "GB"}], "production_countries": [{"iso_3166_1": "GB", "name": "United Kingdom"}, {"iso_3166_1": "US", "name": "United States of America"}], "release_date": "2026-07-15", "revenue": 1684432000, "runtime": 173, "software": false, "spoken_languages": [{"english_name": "English", "iso_639_1": "en", "name": "English"}], "status": "Released", "tagline": "D\u00e9fiez les dieux.", "title": "L'Odyssee", "video": false, "vote_average": 8.009, "vote_count": 3707}
```